



INDIA.RUNS

Build what next India runs on

Team Name : Agilus

Team Leader Name : Soumy Dhiran

Problem Statement : Build an AI system that ranks candidates for a role the way a great recruiter would — by understanding the job and the whole candidate profile, not by matching keywords.

Solution Overview

- **Proposed solution.** A hybrid candidate-ranking engine that fuses three signals — an offline semantic match between the JD and each candidate's real text, a structured evidence-weighted fit score, and a behavioural-availability modifier — and forces impossible / honeypot profiles to the bottom.
- **What makes it different.** It ranks evidence, not keywords. A skill counts only when it appears in real career history, not in a self-listed tag — so keyword-stuffers (a “Marketing Manager” listing every AI skill) are filtered out, while genuine engineers who never used the buzzwords still surface.

JD Understanding & Candidate Evaluation

- **Key requirements from the JD.** Production embeddings & retrieval, vector / hybrid search, ranking evaluation (NDCG / MRR / MAP), strong Python; ~6–8 yrs at product (not services) companies; Pune / Noida or willing to relocate; and explicit negatives — consulting-only careers, CV / speech without NLP, research-only, title-chasers.
- **Signals that matter most.** Real career-history evidence and job title are decisive, then production / shipping language, the experience band, and availability (recruiter response rate and recency).
- **Fit beyond keywords.** Concept evidence is weighted by where it appears — career descriptions count far more than bare skill tags — and the JD's stated negatives apply penalties.

Ranking Methodology

- **Retrieve.** All candidates and a plain-language JD query are vectorised with TF-IDF (1–2 grams); cosine similarity is the semantic signal — fully offline.
- **Score.** A weighted sum of semantic similarity, core-concept evidence, title / role, production signal, experience band, location and nice-to-haves.
- **Combine.** The fit score is multiplied by stated-negative penalties, then by a behavioural modifier (≈ 0.45 – 1.12); honeypots are forced to the bottom. Final sort is score-descending, ties broken by candidate_id.

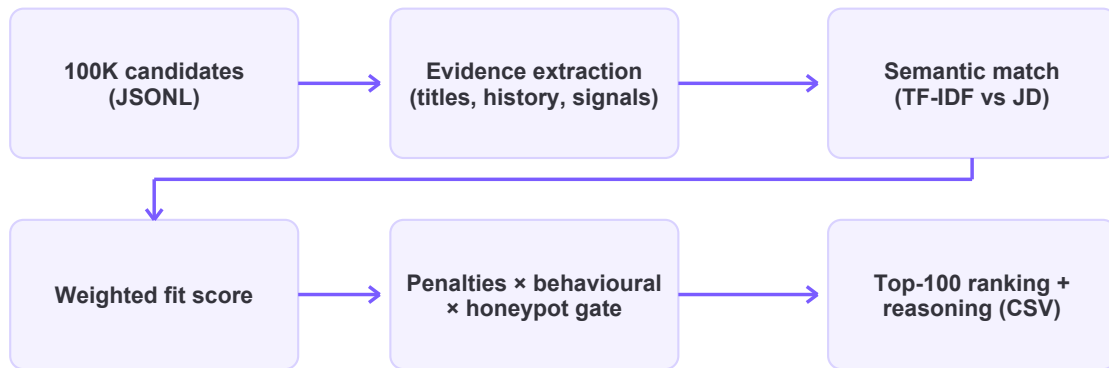
Explainability & Data Validation

- **How decisions are explained.** Every candidate gets a 1–2 sentence reason built only from its own record — title, years, a named in-profile skill, response rate, location — plus honest concerns drawn from the penalties that fired.
- **Preventing hallucination.** Reasoning is assembled from verified fields only; all 100 rows were checked programmatically — every title, years, skill, response rate and location matches the source data.
- **Suspicious profiles.** A detector flags arithmetic-impossible records (a single role longer than the whole career, ≥ 4 'expert' skills with 0 months used) and sinks them; inactive / low-quality profiles are down-weighted by the behavioural modifier.

End-to-End Workflow

- **End-to-end pipeline.** candidates.jsonl → load & extract evidence → TF-IDF semantic match to the JD → weighted fit score → negative penalties → behavioural modifier → honeypot gate → sort → top-100 with reasoning → submission.csv.
- **Validation.** The output passes the official validate_submission.py (100 rows, unique ranks, non-increasing scores) and reproduces identically on every run.

System Architecture



CPU-only · no network / LLM calls · ~60s for 100K candidates · deterministic output

Results & Performance

- **Ranking quality.** The top 100 are 100% AI / ML / NLP / Search / Recommendation engineers with real retrieval and ranking experience — zero non-technical keyword-stuffers, and 34 impossible honeypot profiles gated out of the pool.
- **Meets the constraints.** CPU-only, no network or LLM calls, under 4 GB RAM, and ~60 seconds for all 100,000 candidates (limits are 5 minutes / 16 GB). Fully deterministic — re-running produces a byte-identical CSV (verified by checksum).

Technologies Used

- **Stack.** Python 3 with scikit-learn (TF-IDF + cosine), NumPy and SciPy.
- **Why these.** They are fully offline, CPU-friendly and fast at 100K scale — we deliberately avoid per-candidate neural inference so the system meets real production latency limits, the exact tradeoff the JD asks about.
- **Scope.** pandas / openpyxl are used only for an optional spreadsheet export. No external or hosted-LLM APIs are used anywhere in the ranking.

Submission Assets

- **GitHub repository.** github.com/<your-username>/redrob-candidate-ranker (replace with your link)
- **Ranked output.** Top-100 candidates as a validated CSV (Agilus.csv).
- **Runnable sandbox.** Google Colab notebook that ranks a 100-candidate sample end-to-end.
- **Demo video.** <add link>



INDIA.RUNS

Build what next India runs on

THANK YOU